INFOSYS

HackWithInfy 2026

Complete Campus Interview Preparation Guide

Full Study Material • Algorithm Notes • Core CS Subjects • Interview Q&A • Last-Exam Revision Notes

Prepared by: Mr. S. Sanyasi Rao
Assistant Professor
Vignan's Institute of Information Technology, Visakhapatnam
March 2026

Table of Contents

Table of Contents 2
PART A: Infosys HackWithInfy 2026 – Overview 6
A1. About Infosys HackWithInfy 6
A2. Job Roles & Compensation 6
Detailed Role Comparison 6
A3. HWI 2026 Official Timeline 7
A4. Selection Process – Key Steps 7
A5. Test Formats at a Glance 7
PART B: Data Structures – Complete Study Material 8
B1. Arrays & Strings 8
Core Concepts 8
Complexity Summary 8
Key Patterns & Algorithms 8
Kadane's Algorithm (Maximum Subarray) 9
Sliding Window – Maximum Sum Subarray of Size K 9
Two Pointers – Two Sum in Sorted Array 9
Must-Know String Techniques 9
B2. Linked Lists 10
Types 10
Core Algorithms 10
Reverse Linked List (Code) 10
LRU Cache Implementation 10
B3. Stacks & Queues 12
Key Stack Algorithms 12
Sliding Window Maximum (Deque) 12
B4. Trees & Binary Trees 13
Key Definitions 13
Traversals 13
BFS Level Order Traversal 13
Lowest Common Ancestor (LCA) 14
Heaps (Priority Queues) 14
B5. Graphs 15
Graph Representations 15
Graph Traversals 15
BFS – Shortest Path 15
Topological Sort (DFS-based) 15
Dijkstra's Algorithm (Shortest Path, Weighted) 16

Key Graph Algorithms Comparison 16
PART C: Algorithms – Complete Study Material 17
C1. Sorting Algorithms 17
When to Use Which Sort 17
Merge Sort 18
Quick Sort (with Lomuto partition) 18
C2. Binary Search & Variants 19
Binary Search Variants 19
C3. Dynamic Programming (DP) 20
DP Pattern 1 – Linear DP 20
DP Pattern 2 – 0/1 Knapsack 20
DP Pattern 3 – Longest Common Subsequence (LCS) 20
DP Pattern 4 – Longest Increasing Subsequence (LIS) 20
DP Pattern 5 – Coin Change (Minimum Coins) 21
DP Pattern 6 – Edit Distance 21
DP Summary Table 21
C4. Greedy Algorithms 22
C5. Recursion & Backtracking 22
Subsets (Power Set) 22
N-Queens 23
PART D: Core CS Subjects – Complete Study Material 24
D1. Operating Systems (OS) 24
Key Concepts 24
Process States 24
CPU Scheduling Algorithms 24
Deadlock – Coffman Conditions (ALL four must hold) 25
Deadlock Handling Strategies 25
Memory Management 26
D2. Database Management Systems (DBMS) 27
Core Concepts 27
ACID Properties 27
Normalization 27
SQL Quick Reference 28
SQL Joins 28
D3. Computer Networks 29
OSI Model (7 Layers) – Mnemonic: 'Please Do Not Throw Sausage Pizza Away' 29
TCP vs UDP 29
Key Networking Concepts 30

D4. Object-Oriented Programming & Design Patterns 31
Four Pillars of OOP 31
SOLID Principles 31
Key Design Patterns 31
D5. Compiler Design & Automata Theory (CS/IT Focus) 33
Phases of a Compiler 33
Automata Theory – Key Models 33
D6. Digital Electronics (All Branches) 34
Number Systems 34
Logic Gates – Summary 34
Flip-Flops 34
PART E: Pre-Placement Interview (PPI) Preparation 35
E1. OS – Likely Interview Questions & Answers 35
E2. DBMS – Likely Interview Questions & Answers 35
E3. Networks – Likely Interview Questions & Answers 36
E4. OOP – Likely Interview Questions & Answers 37
E5. Behavioural / HR Interview – Questions & Model Answers 38
Top 15 HR Questions with Preparation Guidance 38
PART F: Last-Exam Quick Revision Notes & Formula Sheets 39
F1. Big-O Complexity Quick Reference 39
F2. Coding Problem Recognition Patterns 40
F3. OS – Last-Minute Quick Notes 41
Scheduling Formulas 41
Must-Remember OS Facts 41
F4. DBMS – Last-Minute Quick Notes 41
Must-Remember DBMS Facts 41
SQL Aggregate Functions Quick Ref 42
F5. Networks – Last-Minute Quick Notes 43
Port Numbers to Remember 43
Subnet Quick Reference 43
F6. OOP & Java/Python – Last-Minute Quick Notes 44
Key Java vs Python Quick Facts 44
F7. Exam Day Strategy – Round 1 & Round 2 45
Before the Test (Night Before) 45
During the Test (Round 1 – Virtual) 45
During the Test (Round 2 – Physical) 45
Critical Edge Cases to Always Check 45
F8. Must-Solve 50 LeetCode Problems Before Round 1 45

Easy (Master all of these first) 45
Medium (Target at least 20 of these) 46
Hard (For SP L2/L3 targeting) 47
F9. Career Trajectory & Technology Focus – Quick Notes 48
Infosys Technology Focus Areas — What to Mention in Interview 48
The 30-Day Preparation Calendar 48
Daily Study Routine (Recommended) 48
F10. Common Mistakes That Cost Marks – Avoid These 49

PART A: Infosys HackWithInfy 2026 – Overview

A1. About Infosys HackWithInfy

HackWithInfy (HWI) is Infosys' premier campus hiring programme for engineering graduates. It identifies the sharpest coding talent across India and offers direct placement into specialized technical roles. HWI 2026 is open to final-year students of B.E / B.Tech / M.E / M.Tech / Dual Degree / MS (Research) / MCA from all disciplines.

Key Fact: No academic criteria (CGPA cutoff) — only coding performance matters!
Condition: NO ACTIVE BACKLOG on the date of joining Infosys.
All eligible streams: CS, IT, ECE, EEE, Mechanical, Civil, Chemical and all others.

A2. Job Roles & Compensation

Digital Specialist Engineer (DSE)	Specialist Programmer L1	Specialist Programmer L2	Specialist Programmer L3
INR 7 LPA	**INR 11 LPA**	**INR 16 LPA**	**INR 21 LPA**
6.25 LPA + Rs.75,000 Joining Bonus	10 LPA + Rs.1 Lakh Joining Bonus	—	—

Detailed Role Comparison

Aspect	DSE	SP L1	SP L2	SP L3
Role Focus	Full-stack; DevSecOps; cross-BU projects	Modern frameworks; AI in SW; Agile DevSecOps	Scalable systems; PoCs & MVPs; AI Master	Enterprise platforms; strategic AI/ML innovation
Key Responsibility	Design, develop, test across lifecycle	Build apps, leverage Cloud/AI, open-source	Solve complex problems; mentor; white papers	Drive architecture; patents; build SLMs
Innovation	Agile delivery	Open-source contributions	Open-source + PoCs + MVPs	Patents + white papers + emerging tech
Complexity	Specialized across BUs	Entry-level niche programming	Intermediate, deep tech expertise	Advanced, strategic, client-facing
Ideal Candidate	Full-stack + DevSecOps mindset	Strong coder; competitive programming	Advanced problem-solver; leadership	Thought leader; innovation-driven

A3. HWI 2026 Official Timeline

Launch	Registrations	Round 1	Round 2	Grand Finale
February 2026	Feb – 15 March 2026	28–29 March 2026	Mid-April 2026	Declared later

A4. Selection Process – Key Steps

STEP 1 – Registration: Fill the official survey form to complete your registration.

STEP 2 – Round 1 (Virtual): 3 coding questions in 3 hours. Easy + Medium + Hard. Any of C, C++, Java, Python, JS.

STEP 3 – Round 2 (Physical): 4 coding questions (attempt 3) in 3 hours. Easy + Medium + Hard + Complex.

STEP 4 – Pre-Placement Interview (PPI): ~1 hour. Technical + Behavioural. Determines final role (DSE vs SP Level).

STEP 5 – Offer Letter: Job offer communication for the trainee role you are selected for.

BONUS: Top 100 from Qualifier tests compete in the HackWithInfy Grand Finale!

A5. Test Formats at a Glance

Parameter	Round 1 (Virtual)	Round 2 (Physical)
Mode	Online / Virtual	In-person at Infosys venue
Questions	3 (all mandatory)	4 (attempt any 3)
Duration	3 hours	3 hours
Difficulty Levels	Easy, Medium, Hard	Easy, Medium, Hard, Complex
Languages	C, C++, Java, Python, JavaScript	C, C++, Java, Python, JavaScript
Top 100	Qualify for Grand Finale	Qualify for Grand Finale

PART B: Data Structures – Complete Study Material

B1. Arrays & Strings

Core Concepts

Term / Concept	Definition & Notes
Array	Contiguous block of memory storing elements of the same type. Indexed from 0 to n-1. Random access in O(1).
Dynamic Array	Resizable array (ArrayList / vector). Amortised O(1) append. Internal array doubles when full.
2D Array	Array of arrays. Access: arr[row][col]. Row-major storage in most languages.
String	Immutable sequence of characters (in Java/Python). Mutable in C/C++. Internally a char array.
StringBuilder	Mutable string buffer for efficient concatenation. Avoids O(n^2) repeated concatenation.

Complexity Summary

Operation / Algorithm	Best	Average	Worst	Space
Access by index	O(1)	O(1)	O(1)	—
Search (unsorted)	O(n)	O(n)	O(n)	—
Search (sorted BS)	O(1)	O(log n)	O(log n)	—
Insert at end	O(1)*	O(1)*	O(n)	O(n) total
Insert at middle	O(n)	O(n)	O(n)	—
Delete at end	O(1)	O(1)	O(1)	—
Delete at middle	O(n)	O(n)	O(n)	—

** Amortised O(1) for dynamic array append*

Key Patterns & Algorithms

- **Two Pointers:** Use left and right pointers moving toward each other. Best for sorted arrays. Pair sum, triplet sum, container with most water. Time: O(n).

- **Sliding Window:** Fixed or variable window over array. Max/min subarray sum, longest substring. Time: O(n).
- **Prefix Sum:** preSum[i] = arr[0]+...+arr[i-1]. Range sum in O(1) after O(n) build. Key for subarray sum problems.
- **Kadane's Algorithm:** Maximum subarray sum. Track current sum; reset to 0 if negative. Time: O(n).
- **Dutch National Flag:** 3-way partition (0s, 1s, 2s) in one pass using lo, mid, hi pointers.
- **Binary Search on Answer:** If a problem asks 'minimum of maximum' or 'maximum of minimum', binary search on the answer space.

Kadane's Algorithm (Maximum Subarray)

```
def max_subarray(arr):
    max_sum = curr_sum = arr[0]
    for x in arr[1:]:
        curr_sum = max(x, curr_sum + x)   # extend or start fresh
        max_sum  = max(max_sum, curr_sum)
    return max_sum
# Time: O(n)  Space: O(1)
```

Sliding Window – Maximum Sum Subarray of Size K

```
def max_sum_k(arr, k):
    window = sum(arr[:k])
    best   = window
    for i in range(k, len(arr)):
        window += arr[i] - arr[i-k]
        best = max(best, window)
    return best
```

Two Pointers – Two Sum in Sorted Array

```
def two_sum_sorted(arr, target):
    l, r = 0, len(arr) - 1
    while l < r:
        s = arr[l] + arr[r]
        if s == target: return (l, r)
        elif s < target: l += 1
        else: r -= 1
    return None
```

Must-Know String Techniques

- **Anagram check:** Sort both strings and compare, OR count character frequencies using a hashmap.
- **Palindrome check:** Two pointers from both ends. Or check s == s[::-1] in Python.
- **KMP Algorithm:** Pattern matching in O(n+m) using failure function (prefix table). Avoids O(nm) brute force.
- **Rabin-Karp:** Rolling hash for pattern matching. Average O(n+m).
- **Z-Algorithm:** Builds Z-array where Z[i] = length of longest substring starting from i that matches prefix of s.

B2. Linked Lists

Types

Type	Description
Singly Linked List	Each node has data + next pointer. Traverse in one direction. O(n) search, O(1) insert at head.
Doubly Linked List	Each node has data + next + prev. O(1) delete if node pointer is known. Used in LRU Cache.
Circular Linked List	Last node points back to head. Useful in round-robin scheduling, Josephus problem.
Linked List vs Array	LL: O(1) insert/delete at known position, O(n) random access. Array: O(1) access, O(n) insert/delete.

Core Algorithms

- **Reverse a Linked List:** Iterative: use prev, curr, next pointers. Recursive: reverse(head.next) then head.next.next = head.
- **Detect Cycle (Floyd's):** Slow pointer moves 1 step, fast pointer moves 2. They meet inside the cycle if one exists. Time O(n), Space O(1).
- **Find Cycle Start:** After detection, reset one pointer to head. Move both 1 step at a time — they meet at cycle start.
- **Middle of Linked List:** Slow/fast pointer: slow=1 step, fast=2 steps. When fast reaches end, slow is at middle.
- **Merge Two Sorted Lists:** Compare heads iteratively, pick smaller, advance that pointer. Time O(m+n).
- **Remove Nth from End:** Two pointers with gap of n. When right reaches end, left.next = left.next.next.

Reverse Linked List (Code)

```
def reverse(head):
    prev, curr = None, head
    while curr:
        nxt       = curr.next
        curr.next = prev
        prev      = curr
        curr      = nxt
    return prev   # new head
```

LRU Cache Implementation

Uses Doubly Linked List + HashMap. Map stores key -> node. DLL maintains access order (most recent at head). O(1) get and put.

```
class LRUCache:
    def __init__(self, cap):
        self.cap   = cap
```

```
        self.cache = {}  # key -> node
        # Dummy head and tail
        self.head, self.tail = Node(0,0), Node(0,0)
        self.head.next = self.tail
        self.tail.prev = self.head

    def get(self, key):
        if key in self.cache:
            self._move_to_front(self.cache[key])
            return self.cache[key].val
        return -1

    def put(self, key, val):
        if key in self.cache: self._remove(self.cache[key])
        node = Node(key, val)
        self._add_front(node)
        self.cache[key] = node
        if len(self.cache) > self.cap:
            lru = self.tail.prev
            self._remove(lru)
            del self.cache[lru.key]
```

B3. Stacks & Queues

Term / Concept	Definition & Notes
Stack	LIFO (Last In First Out). Operations: push, pop, peek — all O(1). Used for: function calls, undo, expression evaluation, DFS.
Queue	FIFO (First In First Out). Enqueue O(1), dequeue O(1) with circular array or linked list. Used for: BFS, scheduling.
Deque	Double-ended queue. Insert/delete at both ends in O(1). Used for: sliding window maximum, palindrome check.
Priority Queue	Heap-backed queue. get-min/max in O(1), insert and extract in O(log n). Used for: Dijkstra's, Kth largest.
Monotonic Stack	Stack maintaining increasing or decreasing order. Used for: next greater element, histogram max rectangle, stock span.

Key Stack Algorithms

- **Balanced Parentheses:** Push opening brackets; on closing bracket check stack top matches. O(n).
- **Next Greater Element:** Traverse right to left maintaining a decreasing monotonic stack. O(n).
- **Largest Rectangle in Histogram:** Use monotonic stack to find left and right boundaries for each bar. O(n).
- **Min Stack:** Maintain an auxiliary stack storing the current minimum. O(1) getMin.
- **Evaluate Postfix Expression:** Push operands; on operator, pop two operands, compute, push result.

Sliding Window Maximum (Deque)

```
from collections import deque
def max_sliding_window(nums, k):
    dq, result = deque(), []
    for i, n in enumerate(nums):
        while dq and dq[0] < i - k + 1: dq.popleft()   # out of window
        while dq and nums[dq[-1]] < n:  dq.pop()        # smaller than n
        dq.append(i)
        if i >= k-1: result.append(nums[dq[0]])
    return result   # Time O(n)  Space O(k)
```

B4. Trees & Binary Trees

Key Definitions

Term / Concept	Definition & Notes
Binary Tree	Each node has at most 2 children (left, right). Root is topmost node. Leaf has no children.
Binary Search Tree (BST)	Left subtree < root < right subtree. Inorder traversal gives sorted sequence. Search/Insert/Delete avg O(log n), worst O(n).
Balanced BST (AVL, Red-Black)	Height difference of subtrees <= 1 (AVL). Guarantees O(log n) for all operations.
Complete Binary Tree	All levels fully filled except last, which fills left to right. Used in heaps.
Perfect Binary Tree	All internal nodes have exactly 2 children and all leaves at same level.
Tree Height	Number of edges on longest path from root to leaf. Height of single node = 0.
Depth of Node	Number of edges from root to that node. Root depth = 0.

Traversals

Traversal	Order	Use Case
Inorder	Left → Root → Right	Gives sorted output for BST
Preorder	Root → Left → Right	Copy tree, prefix expressions
Postorder	Left → Right → Root	Delete tree, postfix expressions
Level Order (BFS)	Level by level using queue	Shortest path in unweighted tree, serialization

BFS Level Order Traversal

```
from collections import deque
def level_order(root):
    if not root: return []
    result, queue = [], deque([root])
    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left:  queue.append(node.left)
            if node.right: queue.append(node.right)
        result.append(level)
    return result
```

Lowest Common Ancestor (LCA)

```
def lca(root, p, q):
    if not root or root == p or root == q: return root
    left  = lca(root.left,  p, q)
    right = lca(root.right, p, q)
    if left and right: return root   # p and q in different subtrees
    return left or right             # both in same subtree
```

Heaps (Priority Queues)

Term / Concept	Definition & Notes
Min-Heap	Parent <= children. Root = minimum element. heapq in Python is a min-heap by default.
Max-Heap	Parent >= children. Root = maximum element. Use -value trick in Python: heapq.heappush(h, -val).
Heapify	Build heap from array in O(n) — more efficient than n insertions which would be O(n log n).
Kth Largest	Use min-heap of size k. If new element > heap top, pop and push. Final top = Kth largest.

B5. Graphs

Graph Representations

Representation	Pros	Cons & Use When
Adjacency Matrix	O(1) edge lookup; simple	O(V^2) space. Use for dense graphs (V^2 ~ E).
Adjacency List	O(V+E) space; efficient traversal	O(degree) edge lookup. Use for sparse graphs.
Edge List	Simple; efficient for sorting edges	Slow lookup. Use for Kruskal's MST.

Graph Traversals

- **BFS (Breadth-First Search):** Uses a queue. Explores level by level. Shortest path in unweighted graph. Time O(V+E). Space O(V).
- **DFS (Depth-First Search):** Uses recursion/stack. Explores as deep as possible before backtracking. Cycle detection, topological sort. Time O(V+E). Space O(V).

BFS – Shortest Path

```
from collections import deque
def bfs(graph, start, end):
    queue   = deque([(start, [start])])
    visited = {start}
    while queue:
        node, path = queue.popleft()
        if node == end: return path
        for nbr in graph[node]:
            if nbr not in visited:
                visited.add(nbr)
                queue.append((nbr, path + [nbr]))
    return []  # no path found
```

Topological Sort (DFS-based)

```
def topo_sort(graph, n):
    visited, stack = set(), []
    def dfs(u):
        visited.add(u)
        for v in graph[u]:
            if v not in visited: dfs(v)
        stack.append(u)
    for i in range(n):
        if i not in visited: dfs(i)
    return stack[::-1]  # reverse of finish order
```

Dijkstra's Algorithm (Shortest Path, Weighted)

```
import heapq
def dijkstra(graph, src, n):
    dist = [float('inf')] * n
    dist[src] = 0
    heap = [(0, src)]    # (distance, node)
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]: continue
        for v, w in graph[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                heapq.heappush(heap, (dist[v], v))
    return dist
# Time: O((V+E) log V)  Does NOT work with negative weights.
```

Key Graph Algorithms Comparison

Algorithm	Purpose	Time	Key Condition
Dijkstra's	Single-source shortest path	O((V+E)logV)	No negative weights
Bellman-Ford	Single-source shortest path	O(VE)	Works with negative weights; detects negative cycles
Floyd-Warshall	All-pairs shortest path	O(V^3)	Works with negative weights (no negative cycles)
Prim's MST	Minimum Spanning Tree	O((V+E)logV)	Works on undirected weighted graphs
Kruskal's MST	Minimum Spanning Tree	O(E log E)	Sort edges; use Union-Find to avoid cycles
BFS	Shortest path (unweighted)	O(V+E)	Unweighted graph only
DFS	Cycle detection, topo sort	O(V+E)	Also finds strongly connected components (Kosaraju's)

PART C: Algorithms – Complete Study Material

C1. Sorting Algorithms

Operation / Algorithm	Best	Average	Worst	Space
Bubble Sort	O(n)	O(n^2)	O(n^2)	O(1)
Selection Sort	O(n^2)	O(n^2)	O(n^2)	O(1)
Insertion Sort	O(n)	O(n^2)	O(n^2)	O(1)
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)
Quick Sort	O(n log n)	O(n log n)	O(n^2)	O(log n)
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)
Counting Sort	O(n+k)	O(n+k)	O(n+k)	O(k)
Radix Sort	O(nk)	O(nk)	O(nk)	O(n+k)
Bucket Sort	O(n+k)	O(n+k)	O(n^2)	O(n+k)

When to Use Which Sort

Scenario	Best Sort Choice
Small array (n < 20)	Insertion Sort — low overhead, cache-friendly, adaptive
General purpose	Merge Sort or Quick Sort — O(n log n) average, well-studied
Memory constrained	Heap Sort — in-place O(n log n), no extra memory
Data nearly sorted	Insertion Sort — nearly O(n) for almost-sorted input
Integer data, small range	Counting Sort — linear time O(n+k)
Stability required	Merge Sort, Counting Sort, Timsort (Python default)
External sort (disk)	Merge Sort — can sort in chunks and merge

Merge Sort

```
def merge_sort(arr):
    if len(arr) <= 1: return arr
    mid   = len(arr) // 2
    left  = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(l, r):
    result, i, j = [], 0, 0
    while i < len(l) and j < len(r):
        if l[i] <= r[j]: result.append(l[i]); i += 1
        else:             result.append(r[j]); j += 1
    return result + l[i:] + r[j:]
# Stable | Time O(n log n) | Space O(n)
```

Quick Sort (with Lomuto partition)

```
def quick_sort(arr, lo=0, hi=None):
    if hi is None: hi = len(arr) - 1
    if lo < hi:
        p = partition(arr, lo, hi)
        quick_sort(arr, lo, p-1)
        quick_sort(arr, p+1, hi)

def partition(arr, lo, hi):
    pivot = arr[hi]
    i = lo - 1
    for j in range(lo, hi):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i+1], arr[hi] = arr[hi], arr[i+1]
    return i + 1
# NOT stable | Time avg O(n log n), worst O(n^2) | Space O(log n) stack
```

C2. Binary Search & Variants

Binary search works on SORTED arrays. Reduces search space by half each step → O(log n).

```
def binary_search(arr, target):
    lo, hi = 0, len(arr) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if arr[mid] == target: return mid
        elif arr[mid] < target: lo = mid + 1
        else: hi = mid - 1
    return -1
```

Binary Search Variants

Variant	Logic / Use Case
First occurrence	Move hi = mid-1 even after match. Return lo when loop ends.
Last occurrence	Move lo = mid+1 even after match. Return hi when loop ends.
Search in rotated array	Find pivot (smallest element) then determine which half to search.
Square root (integer)	Binary search on answer 1..n. Find largest x where x*x <= n.
Find Peak Element	If arr[mid] < arr[mid+1]: peak is on right half (lo = mid+1) else hi = mid.
Minimum in Rotated	Binary search: if arr[mid] > arr[hi], minimum is in right half.
Count occurrences	last_occurrence - first_occurrence + 1.

C3. Dynamic Programming (DP)

DP Formula: Big problem = smaller subproblems + optimal substructure + overlapping subproblems
Approaches: (1) Top-Down Memoization (recursion + cache), (2) Bottom-Up Tabulation (iterative, fill table).

DP Pattern 1 – Linear DP

- **Fibonacci:** dp[i] = dp[i-1] + dp[i-2]. Base: dp[0]=0, dp[1]=1. Can optimize to O(1) space.
- **Climbing Stairs:** dp[i] = dp[i-1] + dp[i-2]. Same as Fibonacci. n steps, 1 or 2 at a time.
- **House Robber:** dp[i] = max(dp[i-1], dp[i-2] + nums[i]). Can't rob adjacent houses.
- **Maximum Product Subarray:** Track both max and min (min * negative = large positive). dp_max[i] = max(nums[i], nums[i]*dp_max[i-1], nums[i]*dp_min[i-1]).

DP Pattern 2 – 0/1 Knapsack

```
def knapsack(weights, values, W):
    n = len(weights)
    dp = [[0]*(W+1) for _ in range(n+1)]
    for i in range(1, n+1):
        for w in range(W+1):
            # Don't take item i
            dp[i][w] = dp[i-1][w]
            # Take item i (if fits)
            if weights[i-1] <= w:
                dp[i][w] = max(dp[i][w],
                               dp[i-1][w - weights[i-1]] + values[i-1])
    return dp[n][W]  # Time O(nW)  Space O(nW) -> can optimize to O(W)
```

DP Pattern 3 – Longest Common Subsequence (LCS)

```
def lcs(s1, s2):
    m, n = len(s1), len(s2)
    dp = [[0]*(n+1) for _ in range(m+1)]
    for i in range(1, m+1):
        for j in range(1, n+1):
            if s1[i-1] == s2[j-1]:
                dp[i][j] = 1 + dp[i-1][j-1]
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    return dp[m][n]  # Time O(mn)  Space O(mn)
```

DP Pattern 4 – Longest Increasing Subsequence (LIS)

```
def lis(nums):
    n = len(nums)
    dp = [1] * n   # each element is LIS of length 1 by itself
    for i in range(1, n):
        for j in range(i):
            if nums[j] < nums[i]:
                dp[i] = max(dp[i], dp[j] + 1)
    return max(dp)  # Time O(n^2). Can do O(n log n) with patience sorting.
```

DP Pattern 5 – Coin Change (Minimum Coins)

```
def coin_change(coins, amount):
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0
    for amt in range(1, amount + 1):
        for c in coins:
            if c <= amt:
                dp[amt] = min(dp[amt], dp[amt - c] + 1)
    return dp[amount] if dp[amount] != float('inf') else -1
# Unbounded Knapsack variant. Time O(amount*len(coins))
```

DP Pattern 6 – Edit Distance

```
def edit_distance(s1, s2):
    m, n = len(s1), len(s2)
    dp = [[0]*(n+1) for _ in range(m+1)]
    for i in range(m+1): dp[i][0] = i   # delete all of s1
    for j in range(n+1): dp[0][j] = j   # insert all of s2
    for i in range(1, m+1):
        for j in range(1, n+1):
            if s1[i-1] == s2[j-1]:
                dp[i][j] = dp[i-1][j-1]            # no operation
            else:
                dp[i][j] = 1 + min(
                    dp[i-1][j],    # delete from s1
                    dp[i][j-1],    # insert into s1
                    dp[i-1][j-1]   # replace
                )
    return dp[m][n]
```

DP Summary Table

Problem	Recurrence	Complexity
0/1 Knapsack	dp[i][w]=max(dp[i-1][w], dp[i-1][w-wi]+vi)	O(nW) time, O(nW) space
Unbounded Knapsack	dp[w]=max over all items of dp[w-wi]+vi	O(nW) time, O(W) space
LCS	if match: dp[i][j]=1+dp[i-1][j-1] else max	O(mn) time, O(mn) space
LIS (n^2)	dp[i]=max(dp[j]+1) for j<i if nums[j]<nums[i]	O(n^2) time, O(n) space
Coin Change	dp[amt]=min(dp[amt-c]+1) for all coins c	O(amount*C) time, O(amount)
Edit Distance	match: dp[i-1][j-1], else 1+min(3 ops)	O(mn) time, O(mn) space
Matrix Chain	dp[i][j]=min(dp[i][k]+dp[k+1][j]+dims)	O(n^3) time, O(n^2) space
Palindrome DP	dp[i][j]=(s[i]==s[j]) and dp[i+1][j-1]	O(n^2) time, O(n^2) space

C4. Greedy Algorithms

Greedy Principle: At each step, make the locally optimal choice hoping it leads to global optimum.
Key: Prove greedy works (exchange argument) or use it for known greedy problems.

Problem	Greedy Strategy	Time
Activity Selection	Sort by finish time, greedily pick non-overlapping activities	O(n log n)
Fractional Knapsack	Sort by value/weight ratio descending; fill greedily	O(n log n)
Huffman Coding	Build min-heap of frequencies; merge two smallest nodes repeatedly	O(n log n)
Job Scheduling (Deadlines)	Sort by profit descending; assign to latest available slot <= deadline	O(n^2)
Minimum Platforms	Sort arrival & departure; use two-pointer to count overlaps	O(n log n)
Gas Station	If total gas >= total cost, solution exists; start where cumulative sum dips below 0	O(n)
Jump Game	Track max reachable index; if current index > max, return false	O(n)
Meeting Rooms II	Min-heap of end times; if earliest end <= start, reuse room	O(n log n)

C5. Recursion & Backtracking

Backtracking = Recursion + Undo (backtrack when a path doesn't lead to a solution).
Template: Choose → Explore → Unchoose (backtrack). Prune early to improve performance.

Subsets (Power Set)

```
def subsets(nums):
    result = []
    def bt(start, current):
        result.append(current[:])    # add copy
        for i in range(start, len(nums)):
            current.append(nums[i]) # choose
            bt(i+1, current)        # explore
            current.pop()           # unchoose
    bt(0, [])
    return result   # Time O(2^n * n)  Space O(n)
```

N-Queens

```
def n_queens(n):
    result, cols = [], set()
    diag1, diag2  = set(), set()

    def bt(row, board):
        if row == n: result.append(board[:]); return
        for col in range(n):
            if col in cols or (row-col) in diag1 or (row+col) in diag2:
                continue
            cols.add(col); diag1.add(row-col); diag2.add(row+col)
            board.append(col)
            bt(row+1, board)
            cols.remove(col); diag1.remove(row-col); diag2.remove(row+col)
            board.pop()
    bt(0, [])
    return len(result)
```

PART D: Core CS Subjects – Complete Study Material

D1. Operating Systems (OS)

Key Concepts

Term / Concept	Definition & Notes
Process	Program in execution. Has its own PCB (Process Control Block): PID, state, registers, memory map, open files.
Thread	Lightweight process. Shares code, data, and heap with parent process. Has its own stack and registers. Faster context switch.
Context Switch	Saving state of current process/thread and loading state of next. Overhead: time spent not doing useful work.
Concurrency	Multiple tasks making progress. Can be interleaved on single CPU. Different from parallelism (truly simultaneous).
Race Condition	Two+ threads access shared data simultaneously and outcome depends on execution order. Prevented by synchronization.
Critical Section	Code segment that accesses shared resources. Must satisfy: Mutual Exclusion, Progress, Bounded Waiting.
Deadlock	Set of processes permanently blocked, each waiting for a resource held by another.
Starvation	Process never gets the resource it needs because others keep taking priority. Solved by aging.

Process States

New → Ready → Running → Waiting → Ready → Running → Terminated
New: process being created. Ready: in queue, waiting for CPU. Running: currently executing. Waiting (Blocked): waiting for I/O or event. Terminated: finished execution.

CPU Scheduling Algorithms

Algorithm	Description	Key Property
FCFS	First Come First Served. Non-preemptive. Simple but poor performance.	Convoy effect: long job blocks short jobs. High average wait time.

SJF (Non-Preemptive)	Shortest Job First. Picks job with shortest burst. Non-preemptive.	Optimal average wait time but requires knowing burst times in advance.
SRTF	Shortest Remaining Time First. Preemptive SJF.	Optimal but high overhead; may starve long jobs.
Round Robin	Each process gets a time quantum; preempted if not done.	Fair; good for time-sharing. Response time depends on quantum size.
Priority	Higher priority processes scheduled first. Can be preemptive or not.	Can cause starvation. Solved by aging (increase priority over time).
Multilevel Queue	Separate queues for different process types. Each queue has own algorithm.	Processes don't move between queues. Foreground vs background.

Deadlock – Coffman Conditions (ALL four must hold)

Condition	Meaning
Mutual Exclusion	At least one resource is held in non-sharable mode.
Hold and Wait	A process holds at least one resource and is waiting for more.
No Preemption	Resources cannot be forcibly taken from a process.
Circular Wait	P1 waits for P2, P2 waits for P3, ..., Pn waits for P1.

Deadlock Handling Strategies

- **Prevention:** Eliminate one of the four conditions. Example: require all resources to be requested at once (eliminates Hold-and-Wait).
- **Avoidance (Banker's Algorithm):** Before allocating, check if system remains in a 'safe state'. Need maximum resource requirement upfront.
- **Detection & Recovery:** Allow deadlock, detect using resource allocation graph, recover by killing processes or preempting resources.
- **Ignorance (Ostrich Algorithm):** Pretend deadlock never occurs. Used in most OS (Windows, UNIX) for performance.

Memory Management

Term / Concept	Definition & Notes
Paging	Divide physical memory into fixed-size frames. Logical memory into same-size pages. Page table maps pages to frames. No external fragmentation.
Segmentation	Divide memory into variable-size segments (code, data, stack). Has external fragmentation. Closer to programmer's view.
Virtual Memory	Allows execution of processes larger than physical RAM. Pages brought in on demand (demand paging).
Page Fault	Accessing a page not in physical memory. OS must load page from disk. If too many = thrashing.
Thrashing	CPU spending more time swapping pages than executing. Fix: reduce multiprogramming degree.
Page Replacement	FIFO, LRU, Optimal (Belady's), Clock (Second Chance). LRU best practical; Optimal theoretical minimum.
TLB	Translation Lookaside Buffer. Hardware cache for page table entries. Speeds up virtual-to-physical translation.

D2. Database Management Systems (DBMS)

Core Concepts

Term / Concept	Definition & Notes
DBMS	Software to manage structured data. Provides concurrent access, security, ACID guarantees.
RDBMS	Relational DBMS. Data stored in tables with rows and columns. SQL for querying.
Schema	Blueprint of database: tables, columns, data types, constraints, relationships.
Primary Key	Uniquely identifies each row. Cannot be NULL. Can be composite (multiple columns).
Foreign Key	Column(s) that reference primary key of another table. Enforces referential integrity.
Index	Data structure (B-tree, Hash) to speed up queries. Speeds read, slows write slightly.
View	Virtual table defined by a SELECT query. Does not store data itself.
Stored Procedure	Pre-compiled SQL code stored in DB. Reduces network traffic, enables code reuse.

ACID Properties

Property	Meaning
Atomicity	Transaction is all-or-nothing. Either all operations commit or all are rolled back.
Consistency	Transaction brings DB from one valid state to another. All constraints satisfied.
Isolation	Concurrent transactions execute as if sequential. Isolation levels: Read Uncommitted, Read Committed, Repeatable Read, Serializable.
Durability	Once committed, transaction survives system failures. Achieved via write-ahead logging.

Normalization

Normal Form	Condition	Example Violation Removed
1NF	Atomic values; no repeating groups; each column has one value	Multi-valued attribute: colors='Red,Blue'

2NF	1NF + no partial dependency (every non-key attr depends on full PK)	In composite PK(A,B): C depends only on A, not AB
3NF	2NF + no transitive dependency (non-key attr depends on non-key attr)	Dept -> DeptName (transitive through Dept_ID)
BCNF	3NF + every determinant is a superkey	Stricter than 3NF; eliminates anomalies 3NF misses

SQL Quick Reference

```
-- DDL (structure)
CREATE TABLE students (id INT PRIMARY KEY, name VARCHAR(50), dept VARCHAR(30));
ALTER TABLE students ADD COLUMN cgpa DECIMAL(4,2);
DROP TABLE students;

-- DML (data)
INSERT INTO students VALUES (1, 'Ravi', 'CSE', 8.5);
UPDATE students SET cgpa=9.0 WHERE id=1;
DELETE FROM students WHERE id=1;

-- DQL (query)
SELECT s.name, d.dept_name
FROM   students s
JOIN   departments d ON s.dept_id = d.dept_id
WHERE  s.cgpa > 8.0
GROUP BY s.dept_id
HAVING COUNT(*) > 5
ORDER BY s.cgpa DESC
LIMIT 10;

-- Subquery
SELECT name FROM students
WHERE cgpa > (SELECT AVG(cgpa) FROM students);
```

SQL Joins

Join Type	Returns
INNER JOIN	Only rows where condition matches in BOTH tables.
LEFT (OUTER) JOIN	All rows from left table + matching rows from right. NULLs for no match on right.
RIGHT (OUTER) JOIN	All rows from right table + matching rows from left. NULLs for no match on left.
FULL OUTER JOIN	All rows from both tables. NULLs where no match on either side.
CROSS JOIN	Cartesian product. Every row of A combined with every row of B. No condition.
SELF JOIN	Table joined with itself. Useful for hierarchical data (employee-manager).

D3. Computer Networks

OSI Model (7 Layers) – Mnemonic: 'Please Do Not Throw Sausage Pizza Away'

Layer	Name	Function	Protocols / Devices
7	**Application**	User-facing services: HTTP, FTP, SMTP, DNS	HTTP, FTP, SMTP, DNS, DHCP
6	**Presentation**	Encryption, compression, data translation	SSL/TLS, JPEG, MPEG
5	**Session**	Establishing, maintaining, terminating sessions	NetBIOS, PPTP, RPC
4	**Transport**	End-to-end communication, error control, flow control	TCP, UDP, port numbers
3	**Network**	Routing, logical addressing	IP, ICMP, OSPF, BGP, Router
2	**Data Link**	Physical addressing (MAC), error detection (CRC), framing	Ethernet, Wi-Fi, Switch, Bridge, MAC
1	**Physical**	Bit transmission over physical medium	Cables, Hubs, Repeaters, Modems

TCP vs UDP

Feature	TCP	UDP
Connection	Connection-oriented (3-way handshake)	Connectionless
Reliability	Reliable — guaranteed delivery, ordering	Unreliable — best effort
Error Checking	Checksums + acknowledgements + retransmission	Checksums only, no retransmission
Speed	Slower (overhead for reliability)	Faster (minimal overhead)
Use Cases	HTTP, FTP, SMTP, SSH, database	DNS, streaming, VoIP, gaming, DHCP
Flow Control	Yes (sliding window)	No
Congestion Control	Yes (slow start, congestion avoidance)	No

Key Networking Concepts

Term / Concept	Definition & Notes
IP Address	IPv4: 32-bit (4 octets, dotted decimal). IPv6: 128-bit (hex). Classes A/B/C/D/E. Private ranges: 10.x, 172.16-31.x, 192.168.x
Subnetting	Dividing a network into smaller sub-networks. Subnet mask determines network vs host portion. CIDR notation: /24 = 255.255.255.0
DNS	Domain Name System. Resolves domain names to IP addresses. Hierarchical: root -> TLD -> authoritative servers. Uses UDP port 53.
DHCP	Dynamic Host Configuration Protocol. Automatically assigns IP address, subnet mask, gateway, DNS to hosts. Uses UDP 67/68.
NAT	Network Address Translation. Maps private IPs to public IP. Allows multiple devices to share one public IP.
HTTP/HTTPS	HTTP: Port 80, stateless, text-based. HTTPS: Port 443, HTTP over TLS/SSL (encrypted). Methods: GET, POST, PUT, DELETE, PATCH.
3-Way Handshake	TCP connection setup: SYN → SYN-ACK → ACK. 4-way teardown: FIN → ACK → FIN → ACK.
OSPF vs BGP	OSPF: Interior Gateway Protocol, link-state, within autonomous system. BGP: Exterior, path-vector, between ASes (internet routing).

D4. Object-Oriented Programming & Design Patterns

Four Pillars of OOP

Pillar	Definition & Example
Encapsulation	Bundling data (attributes) and methods inside a class. Hides internal state using access modifiers (private, protected, public). Example: BankAccount.balance is private; accessed via getBalance().
Abstraction	Hiding implementation details; exposing only what is necessary. Example: Abstract class Shape with method area(). Concrete subclasses Circle, Rectangle implement area().
Inheritance	Child class inherits properties and methods from parent. Promotes code reuse. Python: class Dog(Animal). Java: class Dog extends Animal. Types: single, multiple (via interfaces in Java), multilevel, hierarchical.
Polymorphism	One interface, many implementations. Method Overriding (runtime polymorphism): child overrides parent method. Method Overloading (compile-time): same name, different parameters (Java/C++).

SOLID Principles

Letter	Principle	Meaning
S	**Single Responsibility**	A class should have only one reason to change. One class = one job.
O	**Open/Closed**	Open for extension, closed for modification. Use inheritance/composition to add features.
L	**Liskov Substitution**	Subtypes must be substitutable for their base types without breaking the program.
I	**Interface Segregation**	No client should be forced to depend on methods it doesn't use. Prefer small interfaces.
D	**Dependency Inversion**	High-level modules shouldn't depend on low-level modules. Both depend on abstractions.

Key Design Patterns

Pattern	Category	Intent & Example
Singleton	Creational	One instance only. Private constructor + static instance. Thread-safe via synchronized block or enum.
Factory	Creational	Creates objects without specifying exact class. Factory.create('Circle') returns Circle object.

Builder	Creational	Build complex objects step by step. Avoids telescoping constructors. Pizza.Builder.size(L).crust(thin).build()
Observer	Behavioural	Object (Subject) notifies all dependents (Observers) of state changes. Event-driven. Example: stock price alerts.
Strategy	Behavioural	Define family of algorithms, encapsulate each, make them interchangeable. SortContext.setStrategy(quicksort)
Decorator	Structural	Add responsibilities to objects dynamically. Wrapping: CoffeeWithMilk wraps SimpleCoffee.
Adapter	Structural	Convert interface of a class into one clients expect. Legacy system integration.
Facade	Structural	Simplified interface to complex subsystem. OrderFacade hides inventory, payment, shipping.

D5. Compiler Design & Automata Theory (CS/IT Focus)

Phases of a Compiler

Phase	Input → Output & Description
Lexical Analysis	Source → Tokens. Breaks source code into tokens (identifiers, keywords, literals, operators). Uses regular expressions and DFA.
Syntax Analysis	Tokens → Parse Tree. Checks grammar using Context-Free Grammar (CFG). Methods: LL(1) top-down, LR(0), SLR, LALR, CLR bottom-up.
Semantic Analysis	Parse Tree → Annotated Tree. Type checking, scope resolution, attribute grammar evaluation.
Intermediate Code Gen	Annotated Tree → 3-address code / TAC. Machine-independent representation.
Code Optimization	3AC → Optimized 3AC. Techniques: constant folding, dead code elimination, loop invariant hoisting.
Code Generation	Optimized 3AC → Assembly / Machine Code. Register allocation, instruction selection.

Automata Theory – Key Models

Term / Concept	Definition & Notes
DFA	Deterministic Finite Automaton. For each state and input, exactly one transition. Accepts Regular Languages.
NFA	Non-Deterministic FA. Multiple transitions for same input; epsilon transitions allowed. Equivalent in power to DFA (can be converted).
Regular Language	Described by Regular Expressions or FA. Closed under union, concatenation, star. Pumping Lemma to prove non-regular.
CFG	Context-Free Grammar. Productions: A -> alpha. Describes programming language syntax. Recognized by Pushdown Automata (PDA).
CFL	Context-Free Language. Recognized by PDA. E.g., balanced parentheses, palindromes. Pumping Lemma for CFL to prove not CFL.
Turing Machine	Infinite tape, read/write head, transition function. Most powerful model. Accepts Recursively Enumerable languages.
Decidability	A language is decidable if a TM always halts (yes/no). Halting Problem is undecidable (no TM can decide it for all inputs).

D6. Digital Electronics (All Branches)

Number Systems

Conversion	Method
Decimal → Binary	Divide by 2 repeatedly; remainders (read bottom up). E.g., 13 = 1101(2)
Binary → Decimal	Positional weights: 2^n. E.g., 1101 = 8+4+1 = 13
Hex <-> Binary	Each hex digit = 4 binary bits. F = 1111, A = 1010. E.g., 3F = 0011 1111
1's Complement	Flip all bits. E.g., 0101 -> 1010
2's Complement	1's complement + 1. Used for negative number representation in computers.

Logic Gates – Summary

Gate	Symbol / Boolean	Truth / Key Property
AND	A.B	Output 1 only when ALL inputs are 1.
OR	A+B	Output 1 when ANY input is 1.
NOT (Inverter)	A'	Flips input. 0->1, 1->0.
NAND	(A.B)'	AND + NOT. Universal gate — can implement all gates.
NOR	(A+B)'	OR + NOT. Universal gate — can implement all gates.
XOR	A xor B	Output 1 when inputs are DIFFERENT. A XOR A = 0. A XOR 0 = A.
XNOR	(A xor B)'	Output 1 when inputs are SAME. Used in equality comparators.

Flip-Flops

Term / Concept	Definition & Notes
SR Flip-Flop	Set-Reset. S=1,R=0: Q=1 (Set). S=0,R=1: Q=0 (Reset). S=R=1: INVALID state.
D Flip-Flop	Data/Delay. Q(next) = D on clock edge. Eliminates invalid state of SR. Most common in registers.
JK Flip-Flop	J=K=0: hold. J=1,K=0: set. J=0,K=1: reset. J=K=1: toggle. Universal — no invalid state.
T Flip-Flop	Toggle. T=0: hold. T=1: toggle. Derived from JK with J=K=T. Used in counters.

PART E: Pre-Placement Interview (PPI) Preparation

Interview Duration: ~1 hour | Format: Technical (30-40 min) + Behavioural/HR (20-30 min)
This interview determines whether you get DSE (Rs.7 LPA) or SP L1/L2/L3 (Rs.11-21 LPA).

E1. OS – Likely Interview Questions & Answers

Question	Answer / Key Points
What is the difference between process and thread?	Process: independent execution unit, own memory space, heavy. Thread: lightweight, shares memory with parent process. Context switch between threads faster than between processes.
Explain deadlock with real-life example.	Four Coffman conditions. Real life: two people each holding one chopstick and waiting for the other's chopstick — neither can eat (circular wait).
What is thrashing and how to prevent it?	CPU spends more time swapping pages than executing useful work. Prevention: reduce degree of multiprogramming, use working set model.
Difference between paging and segmentation?	Paging: fixed-size pages, no external fragmentation, programmer unaware. Segmentation: variable-size segments, programmer defines (code/data/stack), external fragmentation possible.
What is a semaphore?	Integer variable used for synchronization. Operations: wait(P) and signal(V). Binary semaphore (0/1) = mutex. Counting semaphore controls access to multiple resources.
What is a mutex vs semaphore?	Mutex: owned by the thread that locked it; only that thread can unlock. Semaphore: not owned; any thread can signal. Mutex for mutual exclusion; semaphore for signaling between threads.
What is virtual memory?	Technique allowing execution of processes larger than RAM. Pages loaded on demand (demand paging). Uses page table + TLB for address translation.

E2. DBMS – Likely Interview Questions & Answers

Question	Answer / Key Points
What is ACID? Give example.	Atomicity: transfer fully completes or not at all. Consistency: balance constraints maintained. Isolation: two transfers don't interfere. Durability: committed data persists even on crash.

Difference between WHERE and HAVING?	WHERE filters rows before grouping. HAVING filters groups after GROUP BY. HAVING can use aggregate functions; WHERE cannot.
What is a clustered vs non-clustered index?	Clustered: data rows stored in order of the indexed column (one per table). Non-clustered: separate structure pointing to data rows (multiple allowed).
What is normalization? Why?	Organizing data to reduce redundancy and improve integrity. Higher normal forms eliminate update/insert/delete anomalies.
What is a transaction? Explain isolation levels.	Set of operations treated as one unit. Isolation levels: Read Uncommitted (dirty reads), Read Committed (no dirty), Repeatable Read (no non-repeatable), Serializable (strictest, no phantom reads).
Difference between DELETE, TRUNCATE, DROP?	DELETE: removes rows, can have WHERE, logged, rollbackable. TRUNCATE: removes all rows, faster, minimal logging, not easily rollbackable. DROP: removes entire table structure.

E3. Networks – Likely Interview Questions & Answers

Question	Answer / Key Points
Explain TCP 3-way handshake.	Client sends SYN. Server responds SYN-ACK. Client sends ACK. Connection established. Purpose: synchronize sequence numbers and confirm both sides can communicate.
What happens when you type www.google.com?	DNS lookup (domain->IP). TCP handshake with server IP. HTTP GET request. Server sends HTML. Browser renders page. (Mention caching, CDN for bonus points.)
Difference between OSI and TCP/IP model?	OSI: 7 layers (theoretical). TCP/IP: 4 layers (Application, Transport, Internet, Network Access). TCP/IP is practical; OSI is reference model.
What is subnetting?	Dividing a network into smaller sub-networks using a subnet mask. E.g., 192.168.1.0/24 gives 256 addresses. /25 splits into two /25 networks of 128 each.
What is ARP?	Address Resolution Protocol. Converts IP address to MAC address within same network segment. Broadcasts 'Who has IP x.x.x.x?' — device with that IP replies with its MAC.
What is a socket?	Endpoint for communication. Combination of IP address + port number. Server socket listens; client socket connects.

E4. OOP – Likely Interview Questions & Answers

Question	Answer / Key Points
What is polymorphism? Types?	One interface, many implementations. Compile-time: method overloading (same name, different params). Runtime: method overriding (child overrides parent method, resolved via vtable).
What is an abstract class vs interface?	Abstract class: can have concrete methods + abstract methods; single inheritance. Interface: only abstract methods (pre-Java 8); multiple inheritance; defines a contract. Use abstract class for base with shared behavior, interface for capability.
What is method overriding vs overloading?	Overriding: same signature in child class, runtime polymorphism (@Override). Overloading: same name, different parameter list, compile-time resolution.
Explain the Singleton pattern.	Ensures only one instance exists. Private constructor, private static instance, public static getInstance() method. Thread-safe: use synchronized or enum-based singleton.
What is composition vs inheritance?	Inheritance: IS-A relationship (Dog IS-A Animal). Composition: HAS-A (Car HAS-A Engine). Prefer composition for flexibility; inheritance for clear IS-A hierarchies.
What is the difference between == and .equals() in Java?	==: reference equality (same object in memory). .equals(): value equality (content comparison, can be overridden). Always use .equals() for String comparison.

E5. Behavioural / HR Interview – Questions & Model Answers

Use the STAR Method: Situation → Task → Action → Result
Be authentic. Prepare 5-6 core stories from your academic/project experience. Quantify outcomes where possible.

Top 15 HR Questions with Preparation Guidance

Question	How to Answer / Key Points
Tell me about yourself.	Structure: Name + degree + branch → Key technical skills (2-3) → Best project (1 line) → What excites you about Infosys / coding. Keep it under 2 minutes. Practice this cold.
Why Infosys?	Research-based answer: mention HWI specifically, Infosys STG, technology focus areas (AI Engineering, Cloud Native), continuous enablement programs. Do NOT say 'brand name or salary'.
Describe your best project.	Use STAR. What problem → Your specific role → Technical choices made → Outcome / what you learned. Mention technologies used.
What are your strengths?	Pick 2-3 relevant strengths: problem-solving, quick learner, coding speed. Give a brief example for each.
What is your biggest weakness?	Be honest but strategic: pick a real weakness you are actively improving. E.g., 'I sometimes over-engineer solutions; I now set a time-box for each feature before coding.'
Describe a challenge you overcame.	Pick a technical or team challenge. Show resilience, collaboration, and technical thinking.
Where do you see yourself in 5 years?	Career goal that aligns with Infosys career trajectory (SP L1 → Senior Technologist path). Mention skill-building: cloud, AI, architecture.
How do you handle failure?	Give real example. Emphasize learning, not dwelling. Show growth mindset.
Tell me about a time you worked in a team.	Describe your specific contribution, how you handled disagreement, how the project succeeded.
Are you comfortable relocating?	If yes, say so clearly. If not, be honest but professional.
What do you know about Infosys business?	IT services, BPO, digital transformation. Key verticals: BFSI, healthcare, retail, manufacturing. Innovation labs. TechForGood initiatives.
How do you keep yourself updated with technology?	LeetCode, competitive programming, GitHub contributions, reading tech blogs (dev.to, Hacker News, IEEE), online courses.
Why should we hire you?	Specific skill + relevant example + enthusiasm. 'I bring X, proven by Y, and I'm excited to contribute to Z.'
Do you have any questions for us?	ALWAYS have 2-3 questions. E.g., 'How does onboarding work for Specialist Programmers?', 'What does the first 6-month training look like?'
What motivates you?	Connect to coding: problem-solving, seeing your code in production, learning new tech, making an impact.

PART F: Last-Exam Quick Revision Notes & Formula Sheets

USE THIS SECTION THE NIGHT BEFORE / MORNING OF THE TEST

Read each table carefully. These are the highest-yield facts and patterns for the Infosys coding test and PPI.

F1. Big-O Complexity Quick Reference

Data Structure	Access	Search	Insert/Delete
Array / Dynamic Array	O(1)	O(n)	O(n) mid / O(1)* end
Linked List (Singly)	O(n)	O(n)	O(1) head / O(n) mid
Hash Map / Hash Set	—	O(1) avg	O(1) avg
Binary Search Tree	O(log n)	O(log n)	O(log n)
Balanced BST (AVL/RB)	O(log n)	O(log n)	O(log n)
Heap (Min/Max)	O(1) peek	O(n)	O(log n)
Stack / Queue	O(1) top	O(n)	O(1)
Trie	—	O(L)	O(L)

Algorithm	Time Complexity	Space
Binary Search	O(log n)	O(1) iter / O(log n) rec
Merge Sort	O(n log n) always	O(n)
Quick Sort	O(n log n) avg / O(n^2) worst	O(log n) stack
Heap Sort	O(n log n) always	O(1)
BFS / DFS (Graph)	O(V+E)	O(V)
Dijkstra's	O((V+E) log V)	O(V)
Bellman-Ford	O(VE)	O(V)
Floyd-Warshall	O(V^3)	O(V^2)
Prim's / Kruskal's MST	O((V+E) log V) / O(E log E)	O(V+E)
0/1 Knapsack (DP)	O(nW)	O(nW) -> O(W)
LCS (DP)	O(mn)	O(mn) -> O(min(m,n))

F2. Coding Problem Recognition Patterns

Read the problem -> Identify the pattern below -> Apply the known algorithm template

When you see...	Think / Use...
Array, find max/min subarray	Kadane's algorithm or Sliding Window
Array, find pair with target sum	Two Pointers (sorted) or Hash Map
Sorted array / search in rotated	Binary Search variant
String anagram/permutation	Character frequency map (hashmap or array[26])
Tree traversal, level-by-level	BFS with a queue
Tree path sum, LCA	DFS recursion
Graph, shortest path, unweighted	BFS
Graph, shortest path, weighted +ve	Dijkstra's with min-heap
Graph, negative weights	Bellman-Ford
Count islands / connected components	DFS/BFS + visited set
Topological ordering, prerequisites	Kahn's BFS or DFS + stack
Detect cycle in directed graph	DFS with 3-color (white/gray/black)
Parentheses matching	Stack
Next greater / smaller element	Monotonic Stack
Top K elements	Min-Heap of size K
Merge K sorted arrays	Max-Heap or merge sort approach
Optimal substructure + overlapping	Dynamic Programming
Choices: take or don't take	0/1 Knapsack DP pattern
All permutations / subsets	Backtracking
Minimize maximum / maximize minimum	Binary Search on answer space
Palindrome substring / sequence	DP on 2D table or Two Pointers (expand center)

F3. OS – Last-Minute Quick Notes

Scheduling Formulas

Turnaround Time = Completion Time - Arrival Time
Waiting Time = Turnaround Time - Burst Time
Response Time = First Run Time - Arrival Time
CPU Utilization (%) = (Burst Time / Turnaround Time) * 100
Throughput = Number of processes completed / Time

Must-Remember OS Facts

- Shortest average waiting time: SJF (non-preemptive). Optimal for average waiting time.
- Optimal page replacement: Replaces page used furthest in future. Theoretical minimum faults.
- LRU page replacement: Replace least recently used. Stack-based, costly to implement exactly.
- Belady's Anomaly: In FIFO, adding more frames can INCREASE page faults (not LRU/Optimal).
- Banker's Algorithm: State is safe if there exists a sequence in which all processes can finish.
- Semaphore: wait(s): if s<=0 block, else s--. signal(s): s++, wake a waiting process.
- Fork() in Linux: Returns 0 to child, child-PID to parent, -1 on error.
- Zombie process: Completed but not reaped by parent (parent hasn't called wait()).
- Orphan process: Parent died before child. Adopted by init (PID 1).

F4. DBMS – Last-Minute Quick Notes

Must-Remember DBMS Facts

- SELECT DISTINCT removes duplicate rows.
- NULL comparisons: use IS NULL / IS NOT NULL, not = NULL.
- GROUP BY must include all non-aggregate columns in SELECT.
- HAVING filters AFTER grouping; WHERE filters BEFORE grouping.
- ORDER BY defaults to ASC. ORDER BY col DESC for descending.
- INNER JOIN = JOIN. Returns only matching rows.
- Functional Dependency X->Y: X uniquely determines Y.
- Candidate Key: minimal superkey. Primary key: chosen candidate key.
- B+ Tree index: leaf nodes contain data/pointers, linked for range queries.
- Deadlock in DBMS: detected via wait-for graph (cycle detection).

SQL Aggregate Functions Quick Ref

Function	Usage
COUNT(*) / COUNT(col)	Count rows / count non-NULL values of column.
SUM(col)	Sum of all non-NULL values.
AVG(col)	Average of non-NULL values.
MAX(col) / MIN(col)	Maximum / minimum value.
GROUP_CONCAT	Concatenate values within group (MySQL). STRING_AGG in PostgreSQL.
RANK() / DENSE_RANK()	Window function: rank within partition. DENSE_RANK has no gaps.
ROW_NUMBER()	Unique sequential number within partition. No ties.

F5. Networks – Last-Minute Quick Notes

Port Numbers to Remember

Port	Protocol	Notes
20/21	**FTP**	20=data, 21=control. Uses TCP.
22	**SSH**	Secure Shell. Encrypted remote login. TCP.
23	**Telnet**	Unencrypted remote login. Replaced by SSH.
25/587	**SMTP**	Email sending. 25 server-to-server, 587 client-to-server.
53	**DNS**	Domain Name System. Both UDP (queries) and TCP (zone transfer).
67/68	**DHCP**	67=server, 68=client. IP address assignment. UDP.
80	**HTTP**	Web traffic. Stateless. TCP.
110	**POP3**	Email retrieval. Downloads and usually deletes from server.
143	**IMAP**	Email retrieval. Syncs and keeps on server.
443	**HTTPS**	HTTP over TLS/SSL. Encrypted web traffic.
3306	**MySQL**	Default MySQL database port.
5432	**PostgreSQL**	Default PostgreSQL port.

Subnet Quick Reference

CIDR	Subnet Mask	Hosts	Use Case
/8	255.0.0.0	16.7M	Class A. Very large networks.
/16	255.255.0.0	65,534	Class B. Large organization networks.
/24	255.255.255.0	254	Class C. Common LAN.
/25	255.255.255.128	126	Split a /24 into two halves.
/30	255.255.255.252	2	Point-to-point links.
/32	255.255.255.255	1 (host route)	Single host identification.

F6. OOP & Java/Python – Last-Minute Quick Notes

Key Java vs Python Quick Facts

Concept	Java	Python
String comparison	equals() method	== (value comparison for strings)
Multiple inheritance	Via interfaces only	Via mixins / multiple base classes
List / ArrayList	ArrayList<Integer> list = new ArrayList<>()	list = []
HashMap	HashMap<K,V> map = new HashMap<>()	dict = {}
Sort	Collections.sort(list)	list.sort() or sorted(list)
Lambda / Functional	list.sort((a,b)->a-b)	list.sort(key=lambda x: x)
Null/None handling	NullPointerException if not checked	if x is None:
Infinite / float max	Integer.MAX_VALUE, Double.MAX_VALUE	float('inf')

F7. Exam Day Strategy – Round 1 & Round 2

Before the Test (Night Before)

1. Review Big-O table and pattern recognition table (F1, F2 above). 30 minutes max.
2. Review 5-7 classic algorithm implementations from memory (reverse LL, BFS, binary search, DP).
3. Sleep 7-8 hours. Your brain consolidates memory during sleep. Critical for performance.
4. Prepare coding environment: IDE, compiler, test cases setup.

During the Test (Round 1 – Virtual)

5. Read all 3 problems in first 10 minutes. Note which is Easy / Medium / Hard.
6. Start with Easy. Solve it completely including edge cases. Score partial credit first.
7. For Medium/Hard: spend 5 minutes planning before coding. Write algorithm in comments first.
8. Test your solution with given examples AND edge cases (empty, single element, max values).
9. If stuck after 40 minutes on a problem: write brute force for partial credit, then move on.
10. Manage time: ~50-60 min per problem is the guideline.

During the Test (Round 2 – Physical)

11. You have 4 questions, must answer 3. Skip the Complex if you are not confident.
12. Solve Easy + Medium fully = guaranteed pass. Hard + anything else = strong shortlisting.
13. You can rough-sketch on paper (since it's physical). Use this to plan before typing.
14. Write clean code with variable names that make sense. Evaluators see your code.
15. Add comments to show your thought process — especially for Hard and Complex questions.

Critical Edge Cases to Always Check

- Empty input: empty array, empty string, n=0
- Single element arrays / single character strings
- Negative numbers (especially in sum/product problems)
- All same elements
- Already sorted / reverse sorted (tests sorting stability and quick sort worst case)
- Integer overflow: use long/long long or Python integers (auto big int)
- Null pointer / None (for tree/linked list problems)
- Disconnected graph (multiple connected components in BFS/DFS)

F8. Must-Solve 50 LeetCode Problems Before Round 1

Easy (Master all of these first)

Problem	Pattern
Two Sum	Hash Map O(n)
Reverse Linked List	Iterative pointer manipulation
Valid Parentheses	Stack

Maximum Subarray	Kadane's Algorithm
Climbing Stairs	DP = Fibonacci
Best Time to Buy & Sell Stock	One-pass min price tracker
Valid Palindrome	Two Pointers
Binary Search	Classic binary search
Merge Two Sorted Lists	Two pointer merge
Invert Binary Tree	Recursion / BFS
Maximum Depth of Binary Tree	Recursive DFS
Missing Number	XOR / sum formula
Move Zeroes	Two pointers in-place
Contains Duplicate	Hash Set O(n)
First Bad Version	Binary Search

Medium (Target at least 20 of these)

Problem	Pattern
Longest Substring Without Repeating	Sliding window + hash set
3Sum	Sort + two pointers
Container With Most Water	Two pointers
Product of Array Except Self	Prefix/suffix products
Find Minimum in Rotated Sorted Array	Binary search
Search in Rotated Sorted Array	Modified binary search
Combination Sum	Backtracking
Subsets	Backtracking
Word Search	DFS + backtracking
Course Schedule	Topological sort (cycle detection)
Number of Islands	BFS/DFS on grid
Clone Graph	BFS + HashMap

Coin Change	DP unbounded knapsack
Unique Paths	DP grid
Longest Increasing Subsequence	DP O(n^2) or O(n log n)
House Robber	DP linear
Jump Game	Greedy
Merge Intervals	Sort + merge
Top K Frequent Elements	Heap / bucket sort
LRU Cache	DLL + HashMap

Hard (For SP L2/L3 targeting)

Problem	Pattern
Trapping Rain Water	Two pointers / monotonic stack
Median of Two Sorted Arrays	Binary search on partition
Word Ladder	BFS shortest path
Serialize & Deserialize Binary Tree	BFS + string parsing
Edit Distance	DP classic
Regular Expression Matching	DP
Merge K Sorted Lists	Priority queue (min-heap)
Largest Rectangle in Histogram	Monotonic stack
Best Time to Buy & Sell Stock III	DP with state machine
N-Queens	Backtracking with pruning
Alien Dictionary	Topological sort
Minimum Window Substring	Sliding window + hashmap
Design Twitter	Heap + OOP design
Find Median from Data Stream	Two heaps (min + max)
Skyline Problem	Heap / segment tree

F9. Career Trajectory & Technology Focus – Quick Notes

Infosys Technology Focus Areas — What to Mention in Interview

Domain	Technologies (Know at least one domain well)
MicroServices	Java, Spring Boot, REST APIs, Docker, Kubernetes, Cloud Foundry
Data Engineering	Kafka, Spark, HDFS, SQL, Python, ETL pipelines
AI Engineering	Machine Learning, Deep Learning, TensorFlow/PyTorch, RAG, Agentic AI
Cloud Native	AWS (EC2, Lambda, S3, RDS), GCP, Azure, Terraform, CI/CD
MEAN/MERN Stack	MongoDB, Express, React, NodeJS, Angular, REST APIs
Emerging Tracks	Kubernetes, Go language, Graph databases, Vector search

The 30-Day Preparation Calendar

Week	Coding Focus	Theory / PPI Prep
Week 1 (Days 1-7)	Arrays, Strings, Sliding Window, Two Pointers, Binary Search (25 problems)	OS: Processes, Threads, Scheduling, Deadlock
Week 2 (Days 8-14)	Linked Lists, Stacks, Queues, Trees, BFS/DFS (25 problems)	DBMS: SQL, Normalization, ACID, Joins
Week 3 (Days 15-21)	Graphs, DP (all patterns), Greedy, Backtracking (25 problems)	Networks: OSI, TCP/UDP, DNS, Subnetting, HTTP
Week 4 (Days 22-28)	Mock tests (3 problems in 3 hrs each day), Hard problems (20)	OOP, Design Patterns, 10 HR questions with STAR stories
Days 29-30	Review weak areas. Light practice only.	Mock PPI interview with peer. Review all quick notes.

Daily Study Routine (Recommended)

16. Morning (1.5 hr): 3-4 LeetCode problems on today's topic — no hints for first 30 minutes.
17. Afternoon (1.5 hr): Theory study — read one chapter, make flashcards or notes.
18. Evening (1 hr): Review yesterday's problems. Identify patterns. Redo ones you got wrong.
19. Night (30 min): Light review of formulas / definitions. No new topics.

F10. Common Mistakes That Cost Marks – Avoid These

Mistake	Correct Approach
Brute force with O(n^2) when O(n) exists	Always ask: can I use a hashmap/two pointers to reduce?
Not considering negative numbers in DP	Handle negatives explicitly (track both max and min running values)
Off-by-one errors in binary search	Use lo<=hi with lo=mid+1 / hi=mid-1 consistently
Stack overflow in deep recursion (Python limit=1000)	Convert to iterative, or increase recursion limit sys.setrecursionlimit()
Mutating array while iterating	Iterate over a copy or use two-pointer / in-place technique
Forgetting visited set in BFS/DFS on graphs	Always maintain visited before pushing to queue/stack
Integer overflow in Java/C++	Use long or check bounds before arithmetic
Not handling disconnected graphs	Outer loop through all nodes calling BFS/DFS if not visited
Wrong result due to float precision	Use integer arithmetic where possible; use eps for float comparison
Empty collection edge case	Check len(arr)==0 or root==None at very start of function

FINAL CHECKLIST — Day of the Exam

[ ] Photo ID / College ID ready (for Round 2 physical)
[ ] Registration email / confirmation printed or on phone
[ ] Coding environment tested (IDE + compiler + templates ready)
[ ] Big-O table reviewed (F1)
[ ] Pattern recognition table reviewed (F2)
[ ] 2-minute 'Tell me about yourself' practiced out loud
[ ] 3 STAR stories prepared for behavioural questions
[ ] Slept 7-8 hours. Ate properly. Arrived 15 min early.